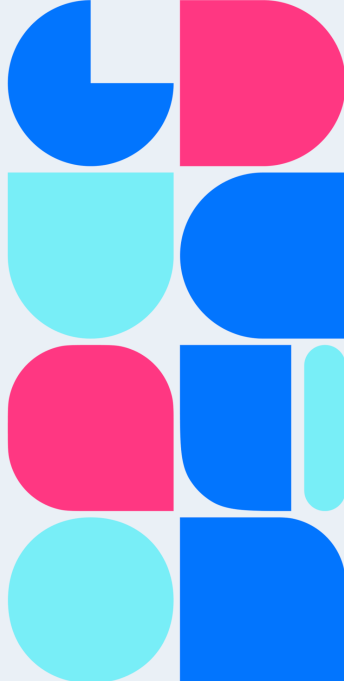




Симулятор

Николай Анохин



Цель симулятора

Зачем нужен симулятор

- Приблизить поведение **реального пользователя**, а не только offline-метрики на готовом логе
- Замкнуть цикл рекомендация → реакция → следующая рекомендация
- Проверять эффект обновлений рекомендательного сервиса

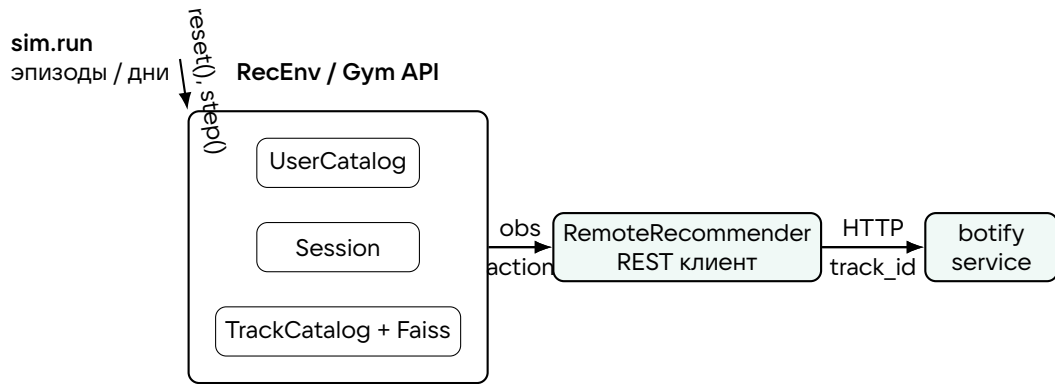
Главная идея

$$\hat{r}_t \Rightarrow \text{feedback}_t \Rightarrow \hat{r}_{t+1}$$

Оценивать нужно не один айтем, а последовательность решений в сессии.

- Симулятор реализован как среда **Gymnasium**
- Запуск разбит на условные **дни**: после каждого дня сервис можно переобучить и перезапустить

Архитектурная схема симулятора



Наблюдение минимально: user id и last track id. Ответ сервиса — следующий трек, а reward рассчитывается внутри симулятора.

Алгоритм работы симулятора I: интересы пользователя

Профиль пользователя

$$u = \{\text{interests}, b, s, B, \gamma\}$$

- interests — список треков-якорей
- b — consume_bias
- s — consume_sharpness
- B — бюджет длины сессии
- γ — штраф за повторы одного артиста

Как стартует сессия

$$t^* \sim \text{Uniform}(\text{interests}_u)$$

$$q_{\text{session}} = e(t^*)$$

- Из интересов случайно выбирается один трек-якорь
- Его эмбединг становится **текущим намерением** сессии
- Первый трек сессии сэмплируется из ближайших соседей этого эмбединга

Алгоритм работы симулятора II: пользовательская сессия

- 1 Botify получает наблюдение $\{user, track\}$ и возвращает следующий трек
- 2 Если трек уже встречался в сессии, время прослушивания равно 0
- 3 Иначе считается близость рекомендации к эмбедингу текущего интереса
- 4 Затем применяется штраф за повторения одного артиста
- 5 Награда = доля прослушивания; сессия завершается при исчерпании бюджета

Формула реакции

$$score = \langle e(i), q_{session} \rangle$$

$$raw_time = \sigma((score - b) \cdot s)$$

$$time = round(raw_time \cdot \gamma^{n_{artist}}, 2)$$

- b управляет порогом "нравится / не нравится"
- s задает резкость отклика
- $\gamma^{n_{artist}}$ штрафует повторения одного артиста

Как готовились данные симулятора

Генерация задачи с помощью LM

- 1 **artists.py**: собирает список популярных артистов по годам, жанрам и странам и сохраняет их метаданные.
- 2 **tracks.py**: для каждого артиста получает треки, очищает и нормализует данные и формирует каталог `tracks.json`.
- 3 **embeddings.py**: по описанию, жанрам, стране и `summary` трека строит его эмбединг и сохраняет общую матрицу векторов.
- 4 **users.py**: на основе каталога треков генерирует синтетических пользователей с интересами и параметрами отклика.

Этот процесс идет от объектов предметной области к представлениям и только затем к пользователям: сначала строится музыкальный каталог, потом его векторное пространство, и уже в нем задаются пользовательские интересы.

Данные симулятора

`https://github.com/anokhin/recsys-course-spring-2026/blob/master/
jupyter/SimulatorData.ipynb`